

Week 4

Learning NLP Basics

Guan-Yun Wang

Department of Information Management

Fu Jen Catholic University

2026.03.20

Playing with Grammar

- Grammar is one of the main building blocks of language. Each human language, and programming language for that matter
- Grammatical rules can be uncovered using NLP and are useful for extracting data from sentences.
- Topics:
 - Counting nouns - plural and singular nouns
 - Getting the dependency parse
 - Extracting noun chunks
 - Extracting the subjects and objects of the sentence
 - Finding patterns in text using grammatical information

Counting nouns - plural and singular nouns

Objective: Determine whether a noun is plural or singular and turn plural nouns into singular, and vice versa.

Thinking:

- * Count the word statistics
- * recognize a word is plural or singular.

Implementation

Getting Ready

- Use `spacy` via two different methods:
 - by looking at the difference between the lemma and the actual word
 - by looking at the `morph` attribute.
- To turn singular nouns into plural or vice versa we will use the `textblob` package.

```
# loading the model from spacy.  
import spacy  
small_model = spacy.load("en_core_web_sm")
```

Identify plural words

Initialize the `text` variable and process it using the `spacy` small model, where `doc` will be a spacy object.

```
text = "I have five birds"    # Or use other sentences to test.  
doc = small_model(text)
```

Here we loop through the `doc` object. The `if` check whether it's a noun and **whether the lemma is the same as the word itself**.

```
for token in doc:  
    if (token.pos_ == "NOUN" and token.lemma_ != token.text):  
        print(token.text, "plural")
```

Function for determining singular and plural

Firstly, we prepare to define a function that returns a tuple, `(noun, number)`. We use an `Enum` class that help us **assign numbers to different values**.

```
from enum import Enum
class Noun_number(Enum):
    SINGULAR = 1
    PLURAL = 2
```

Define the function

```
def get_nouns_number(text, model, method="lemma"):
    nouns = []
    doc = model(text)
    for token in doc:
        if (token.pos_ == "NOUN"):
            if method == "lemma":
                if token.lemma_ != token.text:
                    nouns.append((token.text, Noun_number.PLURAL))
                else:
                    nouns.append((token.text, Noun_number.SINGULAR))
            elif method == "morph":
                if token.morph.get("Number") == "Sing":
                    nouns.append((token.text, Noun_number.PLURAL))
                else:
                    nouns.append((token.text, Noun_number.SINGULAR))
    return nouns
```

Test your code

```
text = "Three geese crossed the road"  
nouns = get_nouns_number(text, small_model, "morph")  
print(nouns)  
nouns = get_nouns_number(text, small_model)  
print(nouns)
```

- Is the result correct?
 - How to solve?
- Try other sentences?

Irregular nouns

Observe the following sentences and test your code.

- The **child** smiled when the teacher gave the **children** new books. 當老師把新書給孩子們(複)時，那位小孩(單)露出了微笑。
- The farmer said **one goose** wandered away from the barn, but the other **geese** stayed close. 農夫說有一隻鵝(單)離開了穀倉，但其他的鵝(複)都待在附近。
- After the hike, her left **foot** was sore, but **both feet** were still strong enough for another climb. 徒步旅行後，她的左腳(單)有些酸痛，但雙腳(複)仍然強壯到可以再爬一次。
- The scientist collected the **leaf** samples and compared them with the **leaves** gathered last year. 科學家收集了葉片(單)樣本，並與去年採集的葉子(複)進行比較。
- Each **woman** in the group shared her findings, and the **women** voted for the best idea. 小組中的每位女性(單)都分享了她的發現，接著所有女性(複)投票選出了最佳想法。

Irregular nouns using large models

In order to analyze irregular nouns, let's try larger models.

You should download the model if you have not yet run this before. (run in your terminal)

```
!python -m spacy download en_core_web_lg
```

Load your model:

```
large_model = spacy.load("en_core_web_lg")
```

Try again:

```
text = "Three geese crossed the road"
nouns = get_nouns_number(text, large_model, "morph")
print(nouns)
nouns = get_nouns_number(text, large_model)
print(nouns)
```

Irregular nouns using Large Language Model

If you remember how to import LLMs, like Mistral AI's models. you can try:

```
# !pip install mistralai
from mistralai.client import Mistral
mistral_client = Mistral(api_key=MISTRAL_API_KEY) # you need to assign one for yourself
model = "mistral-small-2506"
```

Or you can just copy the prompt in the next page and try LLM chatbot to help you.

Test the sentences (LLM prompt)

```
# Testing the sentence by using this prompt

prompt="""Decide whether each noun in the following text is singular or plural.
Return the list in the format of a python tuple: (word, number). Do not provide any additional explanations.
Sentence: Three geese crossed the road."""
response = mistral_client.chat.complete(
    model=model,
    temperature=0,
    max_tokens=256,
    top_p=1.0,
    frequency_penalty=0,
    presence_penalty=0,
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": prompt}
    ],
)
print(response.choices[0].message.content)
```

Compare the results from all methods, and discuss the results with your partners.

Change the nouns from plural to singular

Import the `TextBlob` class from the package:

```
from textblob import TextBlob
```

Initialize a list of text variables and using `TextBlob` class

```
texts = ["book", "goose", "pen", "point", "deer"]  
blob_objs = [TextBlob(text) for text in texts]
```

Pluralize & Singularize

This will show pluralized words:

```
plurals = [blob_obj.words.pluralize()[0] for blob_obj in blob_objs]
print(plurals)
```

This will show singularized words:

```
blob_objs = [TextBlob(text) for text in plurals]
singulars = [blob_obj.words.singularize()[0] for blob_obj in blob_objs]
print(singulars)
```

Getting the dependency parse

A dependency parse is a tool that shows dependencies in a sentence.

The cat wore a hat

- * the root of the senetence is the verb: **wore**
- * the dependents of the sences are the subject, **the cat**, and the object, **a hat**.

- The dependency parse can be very useful in many NLP tasks since it shows the grammatical structure of the sentence, with the subject, the main verb, the object, and so on.
 - In English, the **ROOT** is usually the verb of the sentence.

Implementation

The sentence we're going to take as an example:

```
sentence = 'I have seldom heard him mention her under any other name.'
```

import and load the model

```
#Choose a model you want to use, small or large are both OK
import spacy
small_model = spacy.load("en_core_web_sm")
```


Define a function that will print the word, its grammatical function embedded in the `dep_` attribute, and the explanation of that attribute.

The `dep_` attribute of the `token` object shows the grammatical function of the word in the sentence.

```
def print_dependencies(sentence, model):  
    doc = model(sentence)  
    for token in doc:  
        print(token.text, "\t", token.dep_, "\t", spacy.explain(token.dep_))
```

Check your results:

```
print_dependencies(sentence, small_model)
```

Dependency parse structure

Using the `ancestors` and `children` attributes, we can get the tokens that this token depends on and the tokens that depend on it, respectively.

Print ancestors

```
def print_ancestors(sentence, model):  
    doc = model(sentence)  
    for token in doc:  
        print(token.text, [t.text for t in token.ancestors])
```

```
print_ancestors(sentence, small_model)
```

Observe the results

```
I ['heard']
have ['heard']
seldom ['heard']
heard []
him ['mention', 'heard']
mention ['heard']
her ['mention', 'heard']
under ['mention', 'heard']
any ['name', 'under', 'mention', 'heard']
other ['name', 'under', 'mention', 'heard']
name ['under', 'mention', 'heard']
. ['heard']
```

- `heard` has no ancestors since it is the main word in the sentence.
 - Therefore, all other words depend on it, and in fact, contain `heard` in their ancestor lists.
- The dependency chain can be seen by following the ancestor links for each word.
- A dependency chain will always lead to the root, or the main word, of the sentence.

Print children

We can also create a function that prints out each word and the words that depend on it, its children.

```
def print_children(sentence, model):  
    doc = model(sentence)  
    for token in doc:  
        print(token.text, [t.text for t in token.children])
```

Show the results:

```
print_children(sentence, small_model)
```

Left and Right Children

We can also print left and right children.

- This function will be useful when doing grammatical transformations in the sentence

```
def print_lefts_and_rights(sentence, model):  
    doc = model(sentence)  
    for token in doc:  
        print(token.text, [t.text for t in token.lefts], [t.text for t in token.rights])
```

Show the results:

```
print_lefts_and_rights(sentence, small_model)
```

Results:

The children

```
I []  
have []  
seldom []  
heard ['I', 'have', 'seldom', 'mention', '.']  
him []  
mention ['him', 'her', 'under']  
her []  
under ['name']  
any []  
other []  
name ['any', 'other']  
. []
```

The children (left and right)

```
I [] []  
have [] []  
seldom [] []  
heard ['I', 'have', 'seldom'] ['mention', '.']  
him [] []  
mention ['him'] ['her', 'under']  
her [] []  
under [] ['name']  
any [] []  
other [] []  
name ['any', 'other'] []  
. [] []
```

Subtree

From the subtrees that each word is part of, we can see the grammatical phrases that appear in the sentence, such as the noun phrase, `any other name`, and the prepositional phrase, `under any other name`:

```
def print_subtree(sentence, model):  
    doc = model(sentence)  
    for token in doc:  
        print(token.text, [t.text for t in token.subtree])  
  
print_subtree(sentence, small_model)
```

```
I ['I']  
have ['have']  
seldom ['seldom']  
heard ['I', 'have', 'seldom', 'heard', 'him', 'mention', 'her', 'under', 'any', 'other', 'name', '.']  
him ['him']  
mention ['him', 'mention', 'her', 'under', 'any', 'other', 'name']  
her ['her']  
under ['under', 'any', 'other', 'name']  
any ['any']  
other ['other']  
name ['any', 'other', 'name']  
['.']
```

Can you do the same thing in Chinese?

Try to analyze this sentence:

```
sentence_ch = "我很少聽到他提起她的其他名字。"
```

Hints:

```
small_model_ch = spacy.load("zh_core_web_sm")  
print_dependencies(sentence_ch, small_model_ch)
```

Show the results about the all dependencies, ancestors, children, and subtree. (So you will have at least four results)

Think more about dependency

Easy sentences for dependency parsing:

1. 學生在教室裡閱讀新的課本。
2. 老師昨天在會議中提出一個重要的想法。
3. 研究人員收集資料並分析結果。
4. 他走進教室，準備開始今天的課程。
5. 這位勤奮的學生完成了所有複雜的作業。
6. 圖書館裡的新書吸引了許多好奇的讀者。
7. 機器人在實驗室的桌子下運行測試。
8. 我們在會議結束後討論新的專案方向。
9. 因為資料不完整，所以模型的準確率沒有提升。
10. 他努力準備簡報，但仍然感到有些緊張。
11. 這份報告被團隊中的每個成員仔細檢查。
12. 所有的設備都被工程師重新設定。

Difficult Sentences:

1. 今天星期一。
2. 這個很好吃。
3. 明天早上八點。
4. 有點奇怪。
5. 這邊比較快。
6. 等一下再說。
7. 看不太懂。
8. 忘記帶了。
9. 要不要一起走？
10. 錢包不見了。
11. 後面還有。
12. 差一點點。

Extracting noun chunks

Noun chunks are known in linguistics as noun phrases. They represent nouns and any words that depend on and accompany nouns.

The noun chunks of *The big red apple fell on the scared cat* :

- *the big red apple*
- *the scared cat*

Extracting these noun chunks is instrumental to many other downstream NLP tasks.

- named entity recognition
- processing entities and relations
etc.

Implementation

(noun_chunks cannot be applied on Chinese)

Define the function that will print out the noun chunks:

```
def print_noun_chunks(text, model):  
    doc = model(text)  
    for noun_chunk in doc.noun_chunks:  
        print(noun_chunk.text)
```

Read the text from the `sherlock_holmes_1.txt` file

```
sherlock_holmes_part_of_text = read_text_file("../data/sherlock_holmes_1.txt")  
print_noun_chunks(sherlock_holmes_part_of_text, small_model)
```

Extracting subjects and objects of the sentence

Sometimes, we might need to find the subject and direct objects of the sentence, and that is easily accomplished with the `spacy` package.

Strategy:

- If the **dependency tag** contains `subj`, returns that word should be a subject.
- If the **dependency tag** contains `dobj`, it should be an object.

Get subject and object phrase

subject:

```
def get_subject_phrase(doc):  
    for token in doc:  
        if ("subj" in token.dep_):  
            subtree = list(token.subtree)  
            start = subtree[0].i  
            end = subtree[-1].i + 1  
            return doc[start:end]
```

object:

```
def get_object_phrase(doc):  
    for token in doc:  
        if ("dobj" in token.dep_):  
            subtree = list(token.subtree)  
            start = subtree[0].i  
            end = subtree[-1].i + 1  
            return doc[start:end]
```

Test your code

```
sentences = [  
    "The big black cat stared at the small dog.",  
    "Jane watched her brother in the evenings.",  
    "Laura gave Sam a very interesting book."  
]  
for sentence in sentences:  
    doc = small_model(sentence)  
    subject_phrase = get_subject_phrase(doc)  
    object_phrase = get_object_phrase(doc)  
    print(sentence)  
    print("\tSubject:", subject_phrase)  
    print("\tDirect object:", object_phrase)
```

Can you try Chinese?

If the sentence did not have a direct object or does not have a subject, `None` is printed out.

我很少聽到他提起她的其他名字。

Subject: 我

Direct object: 他

今天是星期一。

Subject: None

Direct object: None

後面還有一個人。

Subject: None

Direct object: 一個人

Finding patterns in text using grammatical information

- We can use the grammatical properties of the words to create the patterns.
- We can specify grammatical patterns.

We will be using the `spaCy Matcher` object to specify and find patterns. It can match different properties, not just grammatical.

```
from spacy.matcher import Matcher  
matcher = Matcher(small_model.vocab)
```


Create a list of patterns and add them to the matcher

The possible examples are written in comments.

```
patterns = [  
    [{"POS": "VERB"}], # paints  
    [{"POS": "AUX"}, {"POS": "VERB"}], # was observing  
    [{"POS": "AUX"}, {"POS": "ADJ"}], # were late  
    [{"POS": "AUX"}, {"POS": "VERB"}, {"POS": "ADP"}] #were staring at  
]  
matcher.add("Verb", patterns)
```

Read the part of the *Sherlock Holmes* text:

```
sherlock_holmes_part_of_text = read_text_file("../data/sherlock_holmes_1.txt")
doc = small_model(sherlock_holmes_part_of_text)
```

Using the `Matcher` object and processed text.

```
matches = matcher(doc)
for match_id, start, end in matches:
    string_id = small_model.vocab.strings[match_id]
    span = doc[start:end]
    print(match_id, string_id, start, end, span.text)
```

See also: using parts of speech, regular expression or other attribute are all allowed. For more information, see the spaCy documentation: <https://spacy.io/usage/rule-based-matching>.